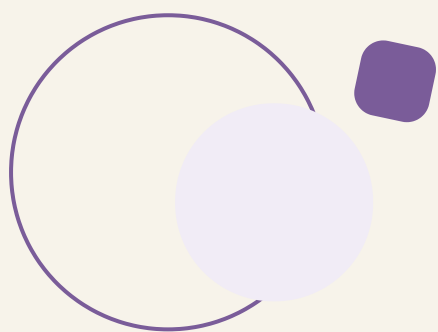




AI 开发 黑话翻译手册

第三册 | AI 编程与 GitHub

程序员是怎么说的，普通人应该怎么理解。

把黑话翻译成成人话，再翻译成 AI 能执行的开发需求。

AI 开发黑话翻译手册

第三册 | AI 编程与 GitHub

书名

AI 开发黑话翻译手册 (AI Development Jargon Translator)

副标题

程序员是怎么说的，普通人应该怎么理解

本册

第三册 · AI 编程与 GitHub · 共 84 个术语

版本

v1.0 · 2026 年

许可

本手册以 CC BY 4.0 协议开源，可自由分享、打印、用于学习与教学，注明出处即可。

本手册面向完全不懂技术的读者。所有解释都尽量用生活语言，不用黑话解释黑话。它同时是 “AI Product Clarifier” 技能的知识底座：当用户表示 “我不太理解” 时，可引用本手册中的解释。

怎么读这本书

这不是词典，是翻译器。每一页把一个技术黑话，翻译成你听得懂的话，再翻译成 AI 能执行的开发需求。

你不需要从头读到尾。遇到听不懂的词，翻到那一页，看一眼「一句话理解」，通常就够了。想更深入，再往下看例子和用法。

每一页长这样

①	分类标签	页面右上角，标明这个词属于产品 / 开发 / 部署 / AI 等哪一类。
②	中文术语	页面最大的字，就是这一页要讲的黑话。
③	英文术语	黑话的英文原文，方便你在真实软件里对照。
④	一句话理解	整页最重要的一句。看不懂别的，看这句就够。
⑤	生活中的例子	用你熟悉的日常事物，帮你秒懂。
⑥	程序员想表达什么	这个词在开发里到底指什么。
⑦	AI 会翻译成什么	你说人话，AI 实际会去做的开发动作。
⑧	什么时候需要 / 不需要	帮你判断这东西你到底要不要。
⑨	没有它会怎样	不做的后果，帮你权衡。
⑩	怎么告诉 AI	可以直接复制给 AI 的一句话。
⑪	记忆口诀	一句话记住它。

翻译逻辑：程序员语言 → 普通人语言 → AI 能执行的开发需求语言。

目录

01	ChatGPT	ChatGPT	001	22	MCP	Model Context Protocol	022
02	Codex	Codex	002	23	RAG	Retrieval-Augmented Gene...	023
03	Claude Code	Claude Code	003	24	Embedding	Embedding	024
04	Cursor	Cursor	004	25	向量数据库	Vector Database	025
05	Gemini	Gemini	005	26	Token	Token	026
06	Windsurf	Windsurf	006	27	多模态	Multimodal	027
07	OpenAI API	OpenAI API	007	28	OCR	OCR	028
08	Anthropic API	Anthropic API	008	29	语音识别	Speech Recognition	029
09	模型	Model	009	30	图像理解	Image Understanding	030
10	大模型	Large Language Model	010	31	代码生成	Code Generation	031
11	Prompt	Prompt	011	32	代码审查	Code Review	032
12	System Prompt	System Prompt	012	33	调试	Debug	033
13	User Prompt	User Prompt	013	34	重构	Refactor	034
14	上下文	Context	014	35	单元测试	Unit Test	035
15	Memory	Memory	015	36	集成测试	Integration Test	036
16	Tool	Tool	016	37	Lint	Lint	037
17	Function Calling	Function Call...	017	38	IDE	IDE	038
18	Reasoning	Reasoning	018	39	VS Code	VS Code	039
19	Agent	Agent	019	40	Markdown	Markdown	040
20	Workflow	Workflow	020	41	GitHub	GitHub	041
21	Skill	Skill	021	42	Repository	Repository	042

目录 (续)

43	Fork Fork	043	64	README README	064
44	Clone Clone	044	65	License License	065
45	Commit Commit	045	66	MIT MIT License	066
46	Branch Branch	046	67	Apache Apache License	067
47	Merge Merge	047	68	开源 Open Source	068
48	Pull Request Pull Request	048	69	闭源 Closed Source	069
49	Issue Issue	049	70	贡献指南 Contributing Guide	070
50	Release Release	050	71	版本号 Version Number	071
51	Tag Tag	051	72	Semantic Version Semantic V... ..	072
52	Star Star	052	73	Roadmap Roadmap	073
53	Watch Watch	053	74	Milestone Milestone	074
54	Contributor Contributor	054	75	Label Label	075
55	Discussion Discussion	055	76	Template Template	076
56	Wiki Wiki	056	77	Boilerplate Boilerplate	077
57	GitHub Actions GitHub Actions	057	78	Scaffold Scaffold	078
58	Diff Diff	058	79	Bug Report Bug Report	079
59	Patch Patch	059	80	Feature Request Feature Req... ..	080
60	Conflict Conflict	060	81	Push Push	081
61	Cherry-pick Cherry-pick	061	82	Pull Pull	082
62	Rebase Rebase	062	83	Remote Remote	083
63	Main Main / Master	063	84	Origin Origin	084

ChatGPT

ChatGPT



一句话理解

一个能陪你聊天、帮你写东西的 AI。

- 生活中的例子

像一个无所不聊、随叫随到的万能助手。

- 程序员真正想表达什么

OpenAI 的对话式大模型产品。

- AI 会翻译成什么开发需求

用自然语言让它帮你写、答、改。

- ✓ 什么时候需要

想快速问答、起草内容时。

- ✗ 什么时候不需要

要精确可靠、不能出错的计算时。

- 没有它会怎样

少了一个随手可用的智能助手。

小白应该怎么告诉 AI

帮我用 ChatGPT 风格的对话来__。

有问题，先问它。

Codex

Codex



一句话理解

专门帮你写代码的 AI。

- 生活中的例子

像一个随叫随到、会各种语言的程序员。

- 程序员真正想表达什么

面向代码生成的模型/工具。

- AI 会翻译成什么开发需求

用它把需求直接变成代码。

- ✓ 什么时候需要

要快速生成或补全代码时。

- ✗ 什么时候不需要

非编程任务时。

- 没有它会怎样

写代码全靠自己，又慢又累。

小白应该怎么告诉 AI

用 Codex 帮我把__写成代码。

‘ 会写代码的 AI 帮手。 ’

Claude Code

Claude Code



一句话理解

能直接在你电脑里帮你写和改代码的 AI 助手。

- 生活中的例子

像请了个程序员坐在你电脑前，边聊边帮你改项目。

- 程序员真正想表达什么

Anthropic 的智能体式编程工具。

- AI 会翻译成什么开发需求

让它读你的项目、直接动手改代码。

- ✓ 什么时候需要

要 AI 真正上手改项目时。

- ✗ 什么时候不需要

只想聊两句、不动代码时。

- 没有它会怎样

AI 只能给建议，不能真正帮你动手。

小白应该怎么告诉 AI

用 Claude Code 帮我在项目里实现__。

“会动手的编程搭子。”

Cursor

Cursor



一句话理解

内置了 AI 的代码编辑器。

- **生活中的例子**

像一支会自动帮你接话的智能笔。

- **程序员真正想表达什么**

深度集成 AI 的开发编辑器。

- **AI 会翻译成什么开发需求**

在编辑器里让 AI 边写边帮你。

- ✓ **什么时候需要**

写代码时想随时叫 AI 时。

- ✗ **什么时候不需要**

不写代码时。

- **没有它会怎样**

写代码时 AI 帮不上手边的忙。

小白应该怎么告诉 AI

用 Cursor，让 AI 在编辑器里帮我写__。

≡ **带 AI 的智能编辑器。** ≡

Gemini

Gemini



一句话理解

谷歌出的 AI 大模型。

- 生活中的例子

谷歌家的“万能助手”。

- 程序员真正想表达什么

Google 的多模态大模型。

- AI 会翻译成什么开发需求

用 Gemini 完成问答、生成等任务。

✓ 什么时候需要

想用谷歌生态的 AI 时。

✗ 什么时候不需要

指定用别家模型时。

- 没有它会怎样

少了一个可选的 AI 大脑。

小白应该怎么告诉 AI

用 Gemini 帮我__。

“谷歌家的 AI 大脑。”

Windsurf

Windsurf



一句话理解

另一款内置 AI 的编程工具。

- **生活中的例子**

和 Cursor 类似的另一支智能笔。

- **程序员真正想表达什么**

AI 辅助的集成开发环境。

- **AI 会翻译成什么开发需求**

在其中让 AI 协助写代码。

- ✓ 什么时候需要**

想试另一种 AI 编程体验时。

- ✗ 什么时候不需要**

已习惯别的工具时。

- **没有它会怎样**

少一个可选的 AI 编程环境。

小白应该怎么告诉 AI

用 Windsurf 让 AI 帮我写__。

﹂ 又一款 AI 编程搭子。 ﹂

OpenAI API

OpenAI API



一句话理解

让你自己的程序也能调用 ChatGPT 的能力。

- 生活中的例子

像给你一根接到“智能大脑”的电话线。

- 程序员真正想表达什么

OpenAI 提供的模型调用接口。

- AI 会翻译成什么开发需求

在代码里调用 OpenAI 接口用模型。

- ✓ 什么时候需要

想把 AI 能力嵌进自己产品时。

- ✗ 什么时候不需要

只在网页里手动聊时。

- 没有它会怎样

自己的程序用不上 AI 能力。

小白应该怎么告诉 AI

帮我用 OpenAI API 让我的程序能调用 AI。

👉 把 AI 大脑接进你的程序。👈

Anthropic API

Anthropic API



一句话理解

让你的程序调用 Claude 的能力。

- **生活中的例子**

另一根接到智能大脑的电话线。

- **程序员真正想表达什么**

Anthropic 提供的模型调用接口。

- **AI 会翻译成什么开发需求**

在代码里调用 Claude 完成任务。

- ✓ **什么时候需要**

想在产品里用 Claude 时。

- ✗ **什么时候不需要**

只手动网页对话时。

- **没有它会怎样**

程序接不上 Claude 的能力。

小白应该怎么告诉 AI

帮我用 Anthropic API 在程序里调用 Claude。

👉 *把 Claude 接进你的程序。* 👈

模型

Model



一句话理解

AI 的“大脑”，学过海量资料后能答会做。

- 生活中的例子

像一个读过很多书、能举一反三的学生。

- 程序员真正想表达什么

训练得到、可推理生成的 AI 系统。

- AI 会翻译成什么开发需求

选一个合适的模型来完成任务。

- ✓ 什么时候需要

凡是要用 AI 能力时。

- ✗ 什么时候不需要

普通固定逻辑就够时。

- 没有它会怎样

没有能理解和生成的智能核心。

小白应该怎么告诉 AI

帮我选个合适的模型来做__。

ˋ 模型，是 AI 的大脑。ˊ

大模型

Large Language Model



一句话理解

特别大、特别能干的 AI 大脑。

- **生活中的例子**

像博览群书、上知天文的超级学霸。

- **程序员真正想表达什么**

参数规模巨大的语言模型。

- **AI 会翻译成什么开发需求**

用大模型处理复杂语言任务。

✓ **什么时候需要**

任务复杂、要强理解力时。

✗ **什么时候不需要**

简单规则能解决时。

- **没有它会怎样**

复杂语言任务处理不好。

小白应该怎么告诉 AI

用一个大模型来理解并处理__。

“越大越博学，也越能干。”

Prompt

Prompt



一句话理解

你给 AI 的那段话，你怎么问决定它怎么答。

- **生活中的例子**

问路问得清楚，别人才好指路。

- **程序员真正想表达什么**

引导模型输出的输入指令。

- **AI 会翻译成什么开发需求**

把需求写成清晰的 prompt 交给模型。

- ✓ **什么时候需要**

每次要 AI 干活时。

- ✗ **什么时候不需要**

不使用 AI 时。

- **没有它会怎样**

问得含糊，AI 答得跑偏。

小白应该怎么告诉 AI

帮我把这个需求写成一段清楚的 prompt。

ˋ 你问得好，它才答得好。ˊ

System Prompt

System Prompt



一句话理解

事先给 AI 定好的“人设和规矩”。

- **生活中的例子**

上班前先交代员工：你的角色是什么、要守什么规矩。

- **程序员真正想表达什么**

设定模型角色与行为的底层指令。

- **AI 会翻译成什么开发需求**

用 system prompt 固定 AI 的角色和风格。

- ✓ **什么时候需要**

想让 AI 稳定扮演某角色时。

- ✗ **什么时候不需要**

临时一次性问答时。

- **没有它会怎样**

AI 每次风格乱变、不受控。

小白应该怎么告诉 AI

帮我写一段 system prompt，让 AI 始终扮演__。

“先立规矩，再干活。”

User Prompt

User Prompt



一句话理解

你每一次具体提出的问题的要求。

- **生活中的例子**

员工上岗后，你每次交办的具体任务。

- **程序员真正想表达什么**

用户每轮输入的具体请求。

- **AI 会翻译成什么开发需求**

把每次具体需求作为 user prompt 发出。

- ✓ **什么时候需要**

每次向 AI 提具体要求时。

- ✗ **什么时候不需要**

不交互时。

- **没有它会怎样**

AI 不知道你这次到底要什么。

小白应该怎么告诉 AI

我的 user prompt 是：帮我__。

每次的具体活儿，就是它。

上下文

Context



一句话理解

AI 当前记得住的那一段对话和资料。

- **生活中的例子**

聊天时对方记得你前面说过啥，才接得上话。

- **程序员真正想表达什么**

模型一次能参考的输入信息范围。

- **AI 会翻译成什么开发需求**

把相关资料放进上下文供模型参考。

✓ **什么时候需要**

要 AI 结合背景连贯回答时。

✗ **什么时候不需要**

无关历史、单次问答时。

- **没有它会怎样**

AI 忘了前文，答得前言不搭后语。

小白应该怎么告诉 AI

把__这些背景放进上下文，让 AI 参考着答。

💡 **记得住前文，才接得上话。** 💡

Memory

Memory



一句话理解

让 AI 跨对话记住关于你的信息。

- **生活中的例子**

熟客再来，店员记得你爱喝什么。

- **程序员真正想表达什么**

跨会话保存并复用的用户信息。

- **AI 会翻译成什么开发需求**

存储关键信息供后续对话调用。

✓ **什么时候需要**

希望 AI 长期记住偏好时。

✗ **什么时候不需要**

一次性、不需记住时。

- **没有它会怎样**

每次都要从头介绍自己。

小白应该怎么告诉 AI

让 AI 记住我的__，以后不用重复说。

“记住你，下次更懂你。”

Tool

Tool



一句话理解

让 AI 除了说话，还能动手用的工具。

- 生活中的例子

给助手配了电话、计算器，他能真去办事。

- 程序员真正想表达什么

模型可调用的外部功能。

- AI 会翻译成什么开发需求

给 AI 配工具让它能查、能算、能操作。

✓ 什么时候需要

要 AI 执行真实操作时。

✗ 什么时候不需要

纯聊天问答时。

- 没有它会怎样

AI 只会空谈，办不了实事。

小白应该怎么告诉 AI

给 AI 配一个能__的工具。

“有工具在手，AI 才能动手。”

Function Calling

Function Calling



一句话理解

AI 判断该用哪个工具、并按格式去调用它。

- 生活中的例子

助手一看要算账，就自己拿起计算器。

- 程序员真正想表达什么

模型结构化调用外部函数的能力。

- AI 会翻译成什么开发需求

定义函数让模型按需自动调用。

- ✓ 什么时候需要

要 AI 自动选用工具时。

- ✗ 什么时候不需要

无外部功能可调时。

- 没有它会怎样

AI 想办事却不知道怎么调工具。

小白应该怎么告诉 AI

让 AI 用 function calling 在需要时自动调用__。

ˋ 该用啥工具，它自己会拿。ˊ

Reasoning

Reasoning



一句话理解

AI 先想一想、一步步推理再回答。

- **生活中的例子**

解应用题先列步骤，而不是张口就报答案。

- **程序员真正想表达什么**

模型进行多步逻辑推演的能力。

- **AI 会翻译成什么开发需求**

让模型先推理再给出结论。

✓ **什么时候需要**

任务复杂、要严谨思考时。

✗ **什么时候不需要**

简单直答就够时。

- **没有它会怎样**

遇到复杂问题容易想当然出错。

小白应该怎么告诉 AI

让 AI 先一步步推理，再给出__的结论。

“先想清楚，再开口。”

Agent

Agent



一句话理解

能自己规划、自己动手完成任务的 AI。

- 生活中的例子

像一个能独立办事的员工，不用你盯着每一步。

- 程序员真正想表达什么

可自主规划并调用工具完成目标的 AI。

- AI 会翻译成什么开发需求

把目标交给 agent，让它自主拆解执行。

- ✓ 什么时候需要

要 AI 独立完成多步任务时。

- ✗ 什么时候不需要

只需一问一答时。

- 没有它会怎样

每一步都要你手把手教。

小白应该怎么告诉 AI

让一个 agent 自主完成__这件事。

≡ 交给它目标，它自己跑完。≡

Workflow

Workflow



一句话理解

把多个步骤串成一条自动流水线。

- **生活中的例子**

工厂流水线，一道接一道自动完成。

- **程序员真正想表达什么**

预定义的多步骤自动执行流程。

- **AI 会翻译成什么开发需求**

把任务拆成固定步骤串成工作流。

- ✓ **什么时候需要**

步骤固定、要可靠重复时。

- ✗ **什么时候不需要**

步骤多变、要灵活判断时。

- **没有它会怎样**

多步任务全靠人工串，慢又易漏。

小白应该怎么告诉 AI

帮我把__拆成一条自动执行的工作流。

≡ **串成流水线，自动往下走。** ≡

Skill

Skill



一句话理解

打包好的一套本领，让 AI 随时能用。

- 生活中的例子

给助手一个工具箱，需要时打开就会用。

- 程序员真正想表达什么

可复用、可加载的能力封装。

- AI 会翻译成什么开发需求

把常用能力做成 skill 供 AI 调用。

✓ 什么时候需要

有反复要用的固定本领时。

✗ 什么时候不需要

一次性任务时。

- 没有它会怎样

同样的本领每次都要从头教。

小白应该怎么告诉 AI

把__做成一个 skill，让 AI 随时能用。

﹂ 本领打包好，随取随用。 ﹂

MCP

Model Context Protocol



一句话理解

让 AI 统一接上各种外部工具和数据的标准接口。

- 生活中的例子

像统一的插座标准，各种电器都能插。

- 程序员真正想表达什么

连接模型与外部工具/数据的开放协议。

- AI 会翻译成什么开发需求

用 MCP 把外部数据和工具接给 AI。

- ✓ 什么时候需要

要给 AI 接多种外部能力时。

- ✗ 什么时候不需要

无需外部数据时。

- 没有它会怎样

每接一个工具都要单独造接口。

小白应该怎么告诉 AI

用 MCP 把__这些工具接给 AI 用。

ˋ 统一插座，什么都能接。ˊ

RAG

Retrieval-Augmented Generation



一句话理解

先查资料再回答，让 AI 有根有据。

- **生活中的例子**

答题前先翻书找到相关那页，再作答。

- **程序员真正想表达什么**

检索外部知识辅助生成的技术。

- **AI 会翻译成什么开发需求**

先检索相关资料，再让模型据此回答。

✓ **什么时候需要**

要 AI 基于你的资料准确回答时。

✗ **什么时候不需要**

凭通用知识就够时。

- **没有它会怎样**

AI 张口就来、容易编造。

小白应该怎么告诉 AI

用 RAG 让 AI 先查我的__资料再回答。

≡ 先翻书，再作答。≡

Embedding

Embedding



一句话理解

把文字变成数字，方便让机器判断“意思像不像”。

- 生活中的例子

给每段话打上“语义坐标”，意思近的坐标也近。

- 程序员真正想表达什么

把内容转成向量表示语义。

- AI 会翻译成什么开发需求

对内容做 embedding 以便相似度检索。

- ✓ 什么时候需要

要按语义搜索、匹配时。

- ✗ 什么时候不需要

只按关键词精确匹配时。

- 没有它会怎样

机器无法判断两段话意思是否相近。

小白应该怎么告诉 AI

帮我把__做成 embedding，方便按意思搜索。

〰 把意思，变成可比的数字。〰

向量数据库

Vector Database



一句话理解

专门存“语义坐标”、按意思找东西的库。

- 生活中的例子

一个能按“意思相近”帮你找资料的仓库。

- 程序员真正想表达什么

存储与检索向量的数据库。

- AI 会翻译成什么开发需求

用向量数据库存 embedding 做语义检索。

✓ 什么时候需要

做 RAG、语义搜索时。

✗ 什么时候不需要

只做关键词精确匹配时。

- 没有它会怎样

海量语义检索又慢又难。

小白应该怎么告诉 AI

用向量数据库存__，支持按意思检索。

“按意思找，不按字面找。”

Token

Token

1 2
3 4

一句话理解

AI 计量文字的最小单位，也用来算钱。

- 生活中的例子

像按字数收费，字越多花得越多。

- 程序员真正想表达什么

模型处理文本的基本计量单元。

- AI 会翻译成什么开发需求

控制输入输出的 token 量以控成本。

- ✓ 什么时候需要

关心成本和长度限制时。

- ✗ 什么时候不需要

不涉及计费 and 长度时。

- 没有它会怎样

内容太长超限或成本失控。

小白应该怎么告诉 AI

帮我精简一下，减少 token 用量。

“AI 按 token 数吃饭。”

多模态

Multimodal



一句话理解

AI 不只懂文字，还能看图、听声。

- **生活中的例子**

一个既会读、又会看、还会听的助手。

- **程序员真正想表达什么**

可处理文本图像音频等多种输入的能力。

- **AI 会翻译成什么开发需求**

用多模态模型处理图文音等内容。

- ✓ **什么时候需要**

要处理图片、语音等时。

- ✗ **什么时候不需要**

只处理纯文字时。

- **没有它会怎样**

只能读字，看不懂图听不懂声。

小白应该怎么告诉 AI

用多模态能力，让 AI 能看懂我发的__。

≡ 又能读，又能看，又能听。≡

OCR

OCR



一句话理解

把图片里的文字认出来变成可复制的字。

- **生活中的例子**

拍张菜单照片，自动变成能复制的文字。

- **程序员真正想表达什么**

从图像中识别文字的技术。

- **AI 会翻译成什么开发需求**

用 OCR 把图片中的文字提取出来。

✓ **什么时候需要**

要从图片/扫描件取文字时。

✗ **什么时候不需要**

本就是可编辑文字时。

- **没有它会怎样**

图里的字只能手动一个个敲。

小白应该怎么告诉 AI

用 OCR 把这张图里的文字提取出来。

≡ **让图片里的字，能被复制。** ≡

语音识别

Speech Recognition



一句话理解

把说的话自动转成文字。

- 生活中的例子

开会时自动把发言记成文字纪要。

- 程序员真正想表达什么

将语音转写为文本的技术。

- AI 会翻译成什么开发需求

用语音识别把音频转成文字。

- ✓ 什么时候需要

要把说话变文字时。

- ✗ 什么时候不需要

只处理文字时。

- 没有它会怎样

录音得靠人一句句听打。

小白应该怎么告诉 AI

用语音识别把这段录音转成文字。

说出来，它写下来。

图像理解

Image Understanding



一句话理解

让 AI 看懂图片里有什么。

- 生活中的例子

发张照片，它能说出“这是只猫在沙发上”。

- 程序员真正想表达什么

对图像内容进行识别与理解的能力。

- AI 会翻译成什么开发需求

用模型分析并描述图片内容。

✓ 什么时候需要

要 AI 分析图片时。

✗ 什么时候不需要

只有文字时。

- 没有它会怎样

AI 面对图片一无所知。

小白应该怎么告诉 AI

让 AI 看懂这张图并告诉我__。

“让 AI 也能看图说话。”

代码生成

Code Generation



一句话理解

把你的要求直接变成代码。

- 生活中的例子

你说“做个登录页”，它就把代码写出来。

- 程序员真正想表达什么

由需求自动产出源代码。

- AI 会翻译成什么开发需求

让 AI 根据描述生成可用代码。

- ✓ 什么时候需要

想快速把想法变代码时。

- ✗ 什么时候不需要

只需讨论方案时。

- 没有它会怎样

每行代码都得自己敲。

小白应该怎么告诉 AI

帮我生成实现__功能的代码。

💡 说需求，出代码。💡

代码审查

Code Review



一句话理解

检查代码有没有问题、能不能更好。

- **生活中的例子**

交作业前让老师帮你挑挑错。

- **程序员真正想表达什么**

对代码质量与缺陷进行检查。

- **AI 会翻译成什么开发需求**

让 AI 或同事审查代码给出改进。

- ✓ **什么时候需要**

代码要合并、保质量时。

- ✗ **什么时候不需要**

随手小改时。

- **没有它会怎样**

问题混进去没人发现。

小白应该怎么告诉 AI

帮我审查这段代码，找出问题和可改进处。

“多一双眼睛，少一个坑。”

调试

Debug



一句话理解

找出程序哪里出错并修好。

- **生活中的例子**

灯不亮，一节节查线路找到坏的那段。

- **程序员真正想表达什么**

定位并修复代码缺陷的过程。

- **AI 会翻译成什么开发需求**

让 AI 帮你定位并修复 bug。

✓ **什么时候需要**

程序出错、行为不对时。

✗ **什么时候不需要**

程序完全正常时。

- **没有它会怎样**

有 bug 却找不到、修不了。

小白应该怎么告诉 AI

帮我调试这段代码，它出现了__问题。

“找到病根，才治得好。”

重构

Refactor

开发



一句话理解

在不改变功能的前提下，把代码整理得更清爽。

- 生活中的例子

东西没扔，只是把房间重新收拾整齐。

- 程序员真正想表达什么

优化代码结构而不改变外部行为。

- AI 会翻译成什么开发需求

让 AI 重构代码提升可读和可维护性。

- ✓ 什么时候需要

代码乱、难维护时。

- ✗ 什么时候不需要

代码已清晰或将废弃时。

- 没有它会怎样

代码越堆越乱，改不动。

小白应该怎么告诉 AI

帮我重构这段代码，功能不变但更清晰。

≡ 不改功能，只理干净。≡

单元测试

Unit Test



一句话理解

单独检验每个小功能对不对。

- **生活中的例子**

菜上桌前，先分别尝尝每样味道。

- **程序员真正想表达什么**

针对最小功能单元的自动化测试。

- **AI 会翻译成什么开发需求**

为关键函数写单元测试。

✓ **什么时候需要**

关键逻辑要保对时。

✗ **什么时候不需要**

一次性小脚本时。

- **没有它会怎样**

小地方出错，攒到最后爆发。

小白应该怎么告诉 AI

帮我给__这个功能写单元测试。

≡ 先测好每一小块。≡

集成测试

Integration Test



一句话理解

检验各部分拼在一起能不能正常配合。

- 生活中的例子

各道菜都好吃，看看整桌搭不搭。

- 程序员真正想表达什么

验证多个模块协同工作的测试。

- AI 会翻译成什么开发需求

写集成测试验证整体流程。

✓ 什么时候需要

多模块协作、要验整体时。

✗ 什么时候不需要

只有单一模块时。

- 没有它会怎样

各自没错，合起来却崩。

小白应该怎么告诉 AI

帮我写集成测试，验证__整个流程能跑通。

“单个都好，还得合得来。”

Lint

Lint



一句话理解

自动检查代码格式和低级毛病。

- **生活中的例子**

写作软件自动标红拼写和格式问题。

- **程序员真正想表达什么**

静态检查代码风格与潜在问题。

- **AI 会翻译成什么开发需求**

配置 lint 自动检查代码规范。

✓ **什么时候需要**

想让代码整齐、少低级错时。

✗ **什么时候不需要**

临时小脚本时。

- **没有它会怎样**

代码风格乱、小毛病多。

小白应该怎么告诉 AI

帮我配好 lint，自动检查代码格式问题。

“小毛病，让工具替你挑。”

IDE

IDE



一句话理解

写代码用的“全能工作台”。

- 生活中的例子

厨师的整体厨房，刀铲锅灶一应俱全。

- 程序员真正想表达什么

集成编辑、运行、调试的开发环境。

- AI 会翻译成什么开发需求

在 IDE 里编写、运行、调试项目。

✓ 什么时候需要

正经写项目时。

✗ 什么时候不需要

改一行小文件时。

- 没有它会怎样

开发效率低，工具东拼西凑。

小白应该怎么告诉 AI

用 IDE 帮我打开并运行这个项目。

“写代码的全能工作台。”

VS Code

VS Code



一句话理解

最流行的免费代码编辑器。

- 生活中的例子

程序员最常用的那把“趁手的笔”。

- 程序员真正想表达什么

轻量、可扩展的主流代码编辑器。

- AI 会翻译成什么开发需求

用 VS Code 编辑和管理代码。

✓ 什么时候需要

日常写代码时。

✗ 什么时候不需要

只看不改时。

- 没有它会怎样

缺一个顺手的写代码工具。

小白应该怎么告诉 AI

用 VS Code 打开项目帮我改__。

“人人都用的顺手编辑器。”

Markdown

Markdown



一句话理解

用简单符号就能排版的轻量写作格式。

- **生活中的例子**

打个 # 就是标题，打 * 就是重点，简单直观。

- **程序员真正想表达什么**

以纯文本标记排版的书写语言。

- **AI 会翻译成什么开发需求**

用 Markdown 写文档和说明。

- ✓ **什么时候需要**

写文档、README、笔记时。

- ✗ **什么时候不需要**

要复杂精细排版时。

- **没有它会怎样**

纯文本没有层次，读着累。

小白应该怎么告诉 AI

用 Markdown 帮我把__整理成带标题的文档。

“几个符号，排出好版。”



一句话理解

全世界程序员放代码、搞协作的大平台。

- **生活中的例子**

像代码界的大型图书馆兼合作社。

- **程序员真正想表达什么**

基于 Git 的代码托管与协作平台。

- **AI 会翻译成什么开发需求**

把项目放到 GitHub 托管和协作。

✓ **什么时候需要**

要托管代码、多人协作、开源时。

✗ **什么时候不需要**

纯本地私人脚本时。

- **没有它会怎样**

代码没处放、没法协作和展示。

小白应该怎么告诉 AI

帮我把这个项目放到 GitHub 上。

🍷 代码的大本营。🍷

Repository

Repository



一句话理解

一个项目在 GitHub 上的“仓库”。

- 生活中的例子

像专门放某个项目所有东西的一个箱子。

- 程序员真正想表达什么

存放项目代码与历史的仓库。

- AI 会翻译成什么开发需求

为项目建一个 repository 管理代码。

✓ 什么时候需要

每个项目都要一个仓库。

✗ 什么时候不需要

无需版本管理时。

- 没有它会怎样

代码散落，无处集中管理。

小白应该怎么告诉 AI

帮我建一个 repository 放这个项目。

“一个项目，一个仓库。”

Fork

Fork



一句话理解

把别人的项目复制一份到自己名下。

- **生活中的例子**

照着别人的菜谱抄一份，改成自己的。

- **程序员真正想表达什么**

复制他人仓库到自己账号。

- **AI 会翻译成什么开发需求**

fork 一份别人的仓库再改。

✓ **什么时候需要**

想基于别人项目改造时。

✗ **什么时候不需要**

自己从零建时。

- **没有它会怎样**

改不了别人的原项目。

小白应该怎么告诉 AI

帮我 fork 这个项目到我的账号。

“抄一份过来，随便改。”

Clone

Clone



一句话理解

把云端的项目下载到自己电脑。

- **生活中的例子**

把图书馆的书复印一份带回家看。

- **程序员真正想表达什么**

把远程仓库完整拉到本地。

- **AI 会翻译成什么开发需求**

clone 仓库到本地进行开发。

- ✓ **什么时候需要**

要在本地改代码时。

- ✗ **什么时候不需要**

只在线看时。

- **没有它会怎样**

本地没有代码，改不了。

小白应该怎么告诉 AI

帮我 clone 这个仓库到本地。

“先拉下来，再动手。”

Commit

Commit



一句话理解

把这次的改动，正式记一笔存档。

- **生活中的例子**

写日记，把今天做的事记下来存好。

- **程序员真正想表达什么**

保存一次代码改动的版本记录。

- **AI 会翻译成什么开发需求**

把改动 commit 成一次带说明的记录。

✓ **什么时候需要**

每完成一小步改动时。

✗ **什么时候不需要**

还没改动时。

- **没有它会怎样**

改动没存档，出事回不去。

小白应该怎么告诉 AI

帮我把这次改动 commit，说明是__。

“改一步，记一笔。”

Branch

Branch



一句话理解

从主线分出一条岔路，单独改不影响别人。

- **生活中的例子**

在正式稿之外另存一份草稿，随便改。

- **程序员真正想表达什么**

并行开发的独立代码线。

- **AI 会翻译成什么开发需求**

开一个 branch 单独做某个功能。

✓ **什么时候需要**

要独立开发某功能、不干扰主线时。

✗ **什么时候不需要**

极简单人改动时。

- **没有它会怎样**

大家挤在主线上互相打架。

小白应该怎么告诉 AI

帮我开一个 branch 来做__功能。

“另开一条道，互不打扰。”

Merge

Merge



一句话理解

把岔路上的改动合回主线。

- **生活中的例子**

把两个人写的稿子合成一份完整稿。

- **程序员真正想表达什么**

将分支改动合并到目标分支。

- **AI 会翻译成什么开发需求**

把功能分支 merge 回主分支。

✓ **什么时候需要**

分支功能做完要并入主线时。

✗ **什么时候不需要**

还在开发中时。

- **没有它会怎样**

改动留在岔路，进不了正式版。

小白应该怎么告诉 AI

帮我把这个分支 merge 回主分支。

“岔路修好，并回主道。”

Pull Request

Pull Request



一句话理解

把你的改动提交上去，请人审核后再合并。

- **生活中的例子**

把改好的稿子交给主编，等他看过再定稿。

- **程序员真正想表达什么**

请求合并改动并供审阅的提案。

- **AI 会翻译成什么开发需求**

开一个 PR 提交改动等待审核合并。

- ✓ **什么时候需要**

多人协作、改动要评审时。

- ✗ **什么时候不需要**

自己单人随便改时。

- **没有它会怎样**

改动没人把关就进主线，风险大。

小白应该怎么告诉 AI

帮我提一个 Pull Request，把__的改动交上去审核。

ˋ 先请人看，再合进去。 ˊ

Issue

Issue



一句话理解

记录一个 bug 或一条建议的“工单”。

- 生活中的例子

报修单：哪坏了、想加啥，都写这。

- 程序员真正想表达什么

跟踪问题或需求的条目。

- AI 会翻译成什么开发需求

用 issue 记录并跟踪问题和需求。

- ✓ 什么时候需要

要收集反馈、管理待办时。

- ✗ 什么时候不需要

无需跟踪时。

- 没有它会怎样

问题和建议记不住、跟丢了。

小白应该怎么告诉 AI

帮我建一个 issue 记录这个__问题。

“有问题，先开张单。”

Release

Release



一句话理解

对外正式发布的一个版本。

- **生活中的例子**

一本书的“第 2 版正式出版”。

- **程序员真正想表达什么**

标记并发布的正式版本。

- **AI 会翻译成什么开发需求**

打一个 release 发布稳定版本。

- ✓ **什么时候需要**

要正式对外发版时。

- ✗ **什么时候不需要**

还在开发时。

- **没有它会怎样**

用户不知道哪个是正式可用版。

小白应该怎么告诉 AI

帮我发一个 release，版本是__。

👉 **正式出版，对外发布。** 👈

Tag

Tag



一句话理解

给某个版本贴的书签，方便随时找回。

- **生活中的例子**

在书里夹个书签，标住重要那一页。

- **程序员真正想表达什么**

标记特定提交的版本标签。

- **AI 会翻译成什么开发需求**

给关键版本打 tag 便于定位。

✓ **什么时候需要**

要标记里程碑版本时。

✗ **什么时候不需要**

无需标版本时。

- **没有它会怎样**

找不回某个重要历史版本。

小白应该怎么告诉 AI

帮我给这个版本打个 tag : __。

📌 夹个书签，好回头找。📌

Star

Star



一句话理解

给喜欢的项目点个赞、收藏起来。

- **生活中的例子**

给喜欢的店点个收藏，下次好找。

- **程序员真正想表达什么**

收藏并表达认可项目的标记。

- **AI 会翻译成什么开发需求**

给项目点 star 收藏关注。

✓ **什么时候需要**

想收藏或表达喜欢时。

✗ **什么时候不需要**

无关注需求时。

- **没有它会怎样**

好项目转头就找不到了。

小白应该怎么告诉 AI

帮我给这个项目点个 star。

‘喜欢就点颗星。’

Watch

Watch



一句话理解

订阅一个项目，有更新就通知你。

- **生活中的例子**

关注某公众号，有新文章就提醒。

- **程序员真正想表达什么**

订阅仓库动态接收通知。

- **AI 会翻译成什么开发需求**

watch 项目以接收更新提醒。

✓ **什么时候需要**

想跟进项目动态时。

✗ **什么时候不需要**

无需跟进时。

- **没有它会怎样**

项目有重要更新你却错过。

小白应该怎么告诉 AI

帮我 watch 这个项目，有更新提醒我。

👁️ **盯着它，有事叫你。**

Contributor

Contributor



一句话理解

为一个项目出过力的人。

- 生活中的例子

一起编了这本书的所有作者。

- 程序员真正想表达什么

向项目提交过贡献的人。

- AI 会翻译成什么开发需求

记录并致谢项目贡献者。

✓ 什么时候需要

开源协作、要致谢时。

✗ 什么时候不需要

纯个人项目时。

- 没有它会怎样

出力的人得不到承认。

小白应该怎么告诉 AI

帮我把贡献者列进项目里。

“出过力的人，都记一功。”

Discussion

Discussion



一句话理解

项目里大家聊天讨论的地方。

- 生活中的例子

项目的“论坛/留言板”。

- 程序员真正想表达什么

围绕项目的开放讨论区。

- AI 会翻译成什么开发需求

开启 discussions 供社区交流。

✓ 什么时候需要

要社区交流、答疑时。

✗ 什么时候不需要

无社区时。

- 没有它会怎样

大家没地方讨论，反馈散乱。

小白应该怎么告诉 AI

帮我开启项目的 discussion 讨论区。

有事大家一起唠。

Wiki

Wiki



一句话理解

项目的“百科全书/文档区”。

- 生活中的例子

项目自带的一本可协作的说明手册。

- 程序员真正想表达什么

项目的协作式文档空间。

- AI 会翻译成什么开发需求

用 wiki 沉淀项目文档。

✓ 什么时候需要

文档多、要系统整理时。

✗ 什么时候不需要

文档极少时。

- 没有它会怎样

文档零散，新人难上手。

小白应该怎么告诉 AI

帮我用 wiki 整理项目的文档。

“项目自己的百科。”

GitHub Actions

GitHub Actions



一句话理解

GitHub 里的自动化机器人，帮你自动干活。

- **生活中的例子**

定好规矩后，它自动帮你测试、发布。

- **程序员真正想表达什么**

GitHub 内置的自动化流水线。

- **AI 会翻译成什么开发需求**

用 Actions 自动测试和部署。

✓ **什么时候需要**

要自动化测试、部署时。

✗ **什么时候不需要**

极简项目时。

- **没有它会怎样**

每次测试部署都得手动。

小白应该怎么告诉 AI

帮我用 GitHub Actions 自动跑测试和部署。

‘定好规矩，它替你干。’

Diff

Diff



一句话理解

看这次和上次比，到底改了哪儿。

- **生活中的例子**

两版文稿对比，红绿标出改动。

- **程序员真正想表达什么**

展示改动差异的对比。

- **AI 会翻译成什么开发需求**

查看 diff 了解具体改动。

✓ **什么时候需要**

评审、确认改了什么时。

✗ **什么时候不需要**

无改动对比需求时。

- **没有它会怎样**

看不清到底动了哪里。

小白应该怎么告诉 AI

帮我看这次的 diff，改了哪些地方。

“一眼看清改了啥。”

Patch

Patch



一句话理解

把一小段改动打包，方便应用到别处。

- **生活中的例子**

一张“补丁贴纸”，贴哪补哪。

- **程序员真正想表达什么**

记录并可应用的改动集合。

- **AI 会翻译成什么开发需求**

生成 patch 应用某段改动。

- ✓ **什么时候需要**

要单独传递某段改动时。

- ✗ **什么时候不需要**

直接合并即可时。

- **没有它会怎样**

小改动难以单独搬运。

小白应该怎么告诉 AI

帮我把这段改动做成 patch。

≡ 哪破补哪的小贴片。≡

Conflict

Conflict



一句话理解

两个人改了同一处，系统不知听谁的。

- **生活中的例子**

两人在同一行写了不同的字，得有人拍板。

- **程序员真正想表达什么**

合并时同处改动的冲突。

- **AI 会翻译成什么开发需求**

解决 merge 时出现的代码冲突。

✓ **什么时候需要**

多人改到同一处时。

✗ **什么时候不需要**

改动互不相干时。

- **没有它会怎样**

改动合不进去，卡住。

小白应该怎么告诉 AI

帮我解决这个合并 conflict，保留__。

💡 **撞车了，得有人定夺。** 💡

Cherry-pick

Cherry-pick



一句话理解

只把某一笔改动单独挑过来用。

- 生活中的例子

从一整桌菜里，只夹你想要的那道。

- 程序员真正想表达什么

选取特定提交应用到当前分支。

- AI 会翻译成什么开发需求

cherry-pick 某个 commit 到本分支。

✓ 什么时候需要

只想要某一笔改动时。

✗ 什么时候不需要

要整体合并时。

- 没有它会怎样

为一笔改动被迫合并一堆。

小白应该怎么告诉 AI

帮我 cherry-pick 那笔__的改动过来。

“只挑想要的那一笔。”

Rebase

Rebase



一句话理解

把你的改动重新接到最新主线上，让历史更整齐。

- **生活中的例子**

把你的稿子接到最新版正文后面，顺成一条线。

- **程序员真正想表达什么**

在新基点上重放提交以整理历史。

- **AI 会翻译成什么开发需求**

用 rebase 把分支变基到最新主线。

- ✓ **什么时候需要**

想让提交历史干净时。

- ✗ **什么时候不需要**

怕搞乱、宁求稳时。

- **没有它会怎样**

提交历史交错混乱。

小白应该怎么告诉 AI

帮我把这个分支 rebase 到最新的主线上。

👉 接到最新，理成一条线。👈

Main

Main / Master



一句话理解

项目的主线，最正式、最可靠的那条。

- **生活中的例子**

一本书的正式定稿版。

- **程序员真正想表达什么**

项目的主分支。

- **AI 会翻译成什么开发需求**

把稳定代码保持在 main 分支。

✓ **什么时候需要**

任何 Git 项目。

✗ **什么时候不需要**

无版本管理时。

- **没有它会怎样**

没有一条可信的主线。

小白应该怎么告诉 AI

把稳定版本保持在 main 分支。

“主线，是最可信的那一条。”

README

README



一句话理解

项目首页的说明书，告诉别人这是什么。

- 生活中的例子

打开项目先看到的“使用说明”。

- 程序员真正想表达什么

介绍项目与用法的入口文档。

- AI 会翻译成什么开发需求

写好 README 说明项目用途和用法。

✓ 什么时候需要

项目要给人看时。

✗ 什么时候不需要

纯私人临时脚本时。

- 没有它会怎样

别人打开项目一脸懵。

小白应该怎么告诉 AI

帮我写一份 README 介绍这个项目。

“先读我，再用我。”

License

License



一句话理解

说明这份代码能不能用、怎么用的许可。

- **生活中的例子**

书上的版权与转载声明。

- **程序员真正想表达什么**

规定使用与分发权利的开源许可。

- **AI 会翻译成什么开发需求**

给项目选一个合适的开源许可证。

✓ **什么时候需要**

开源、要讲清权利时。

✗ **什么时候不需要**

纯私有时。

- **没有它会怎样**

别人不敢用，怕侵权。

小白应该怎么告诉 AI

帮我给项目加一个开源许可证。

👉 **讲清能不能用，别人才敢用。** 👈

MIT

MIT License

FREE

一句话理解

最宽松的开源许可，几乎随使用。

- 生活中的例子

“随便拿去用，出事别怪我”的大方声明。

- 程序员真正想表达什么

极宽松的开源许可证。

- AI 会翻译成什么开发需求

选 MIT 许可让别人自由使用。

- ✓ 什么时候需要

想让项目最大化被使用时。

- ✗ 什么时候不需要

要严格限制使用时。

- 没有它会怎样

别人担心限制，不敢用。

小白应该怎么告诉 AI

帮我给项目用 MIT 许可证。

“随使用，别怪我。”

Apache

Apache License



一句话理解

宽松但更正式、含专利保护的开源许可。

- **生活中的例子**

同样大方，但白纸黑字写得更周全。

- **程序员真正想表达什么**

带专利授权条款的宽松许可证。

- **AI 会翻译成什么开发需求**

选 Apache 许可兼顾开放与保护。

✓ **什么时候需要**

开源又想要专利保障时。

✗ **什么时候不需要**

只要极简许可时。

- **没有它会怎样**

缺少专利层面的保障。

小白应该怎么告诉 AI

帮我给项目用 Apache 许可证。

‘大方之余，多层保障。’

开源

Open Source



一句话理解

把代码公开，谁都能看、能用、能改。

- **生活中的例子**

把菜谱公开，人人可照做、可改良。

- **程序员真正想表达什么**

公开源代码供他人使用与协作。

- **AI 会翻译成什么开发需求**

以开源方式公开并协作维护项目。

✓ **什么时候需要**

想让大家共建、共享时。

✗ **什么时候不需要**

涉及商业机密时。

- **没有它会怎样**

好东西无法被更多人用和改进。

小白应该怎么告诉 AI

帮我把这个项目做成开源的。

“公开出来，众人拾柴。”

闭源

Closed Source



一句话理解

代码不公开，只有自己人能看。

- **生活中的例子**

祖传秘方，只传自家人。

- **程序员真正想表达什么**

不公开源代码的方式。

- **AI 会翻译成什么开发需求**

以闭源方式保护核心代码。

✓ **什么时候需要**

涉及商业机密、要保护时。

✗ **什么时候不需要**

想开放共建时。

- **没有它会怎样**

核心技术容易被照抄。

小白应该怎么告诉 AI

这个项目保持闭源，不公开代码。

“秘方在手，不外传。”

贡献指南

Contributing Guide



一句话理解

告诉别人怎么参与、怎么提交改动。

- 生活中的例子

投稿须知：格式、流程、注意事项。

- 程序员真正想表达什么

指导他人如何贡献的说明文档。

- AI 会翻译成什么开发需求

写 CONTRIBUTING 说明参与方式。

- ✓ 什么时候需要

开源、想让人顺利参与时。

- ✗ 什么时候不需要

纯个人项目时。

- 没有它会怎样

想帮忙的人不知从何下手。

小白应该怎么告诉 AI

帮我写一份贡献指南，说明怎么参与。

‘ 先看须知，再来投稿。 ’

版本号

Version Number

1 2
3 4

一句话理解

用数字标明这是第几版。

- 生活中的例子

软件的“1.2.3”这种编号。

- 程序员真正想表达什么

标识软件版本的编号。

- AI 会翻译成什么开发需求

为每次发布分配版本号。

✓ 什么时候需要

项目会持续更新时。

✗ 什么时候不需要

一次性小东西时。

- 没有它会怎样

分不清谁新谁旧。

小白应该怎么告诉 AI

帮我给这次发布定一个版本号。

“一串数字，说清第几版。”

Semantic Version

Semantic Versioning



一句话理解

一套“大改中改小改”都有含义的版本号规则。

- 生活中的例子

1.0.0 里三段数字，各代表改动大小。

- 程序员真正想表达什么

以主.次.修表意的版本规范。

- AI 会翻译成什么开发需求

按语义化版本规则升级号码。

- ✓ 什么时候需要

想让版本号有意义、可预期时。

- ✗ 什么时候不需要

随手编号即可时。

- 没有它会怎样

版本号乱跳，看不出改了多大。

小白应该怎么告诉 AI

帮我按语义化版本规则给项目定号。

“看号就知改了多大。”

Roadmap

Roadmap



一句话理解

项目未来打算做什么的计划图。

- 生活中的例子

旅行的行程表：接下来去哪。

- 程序员真正想表达什么

按阶段规划功能与目标的蓝图。

- AI 会翻译成什么开发需求

整理一份项目路线图。

- ✓ 什么时候需要

要展示未来方向时。

- ✗ 什么时候不需要

只做眼前一件事时。

- 没有它会怎样

大家看不清项目往哪走。

小白应该怎么告诉 AI

帮我整理一份项目 roadmap。

“有图在手，方向不丢。”

Milestone

Milestone



一句话理解

一个阶段性的重要目标节点。

- **生活中的例子**

跑步的“10 公里”打卡点。

- **程序员真正想表达什么**

标志阶段成果的目标节点。

- **AI 会翻译成什么开发需求**

用 milestone 组织阶段目标。

✓ **什么时候需要**

项目大、要分段推进时。

✗ **什么时候不需要**

任务很短时。

- **没有它会怎样**

干到底没个盼头。

小白应该怎么告诉 AI

把任务分成几个 milestone。

“一个个节点，走成一条路。”

Label

Label



一句话理解

给 issue 或 PR 贴的分类标签。

- 生活中的例子

给便签分“紧急/一般”贴不同颜色。

- 程序员真正想表达什么

用于分类标记条目的标签。

- AI 会翻译成什么开发需求

用 label 给 issue 分类。

✓ 什么时候需要

问题多、要归类时。

✗ 什么时候不需要

条目极少时。

- 没有它会怎样

一堆问题分不清轻重缓急。

小白应该怎么告诉 AI

帮我给这些 issue 打上 label 分类。

“贴个标，好归类。”

Template

Template



一句话理解

填空式的现成模板，照着填就行。

- **生活中的例子**

现成的简历模板，套进去改改。

- **程序员真正想表达什么**

预设结构供复用的模板。

- **AI 会翻译成什么开发需求**

用模板统一 issue/PR 或项目结构。

✓ **什么时候需要**

有重复格式要统一时。

✗ **什么时候不需要**

一次性内容时。

- **没有它会怎样**

每次从零写，格式还乱。

小白应该怎么告诉 AI

帮我做一个__模板，以后照着填。

“有模板，照着填就行。”

Boilerplate

Boilerplate



一句话理解

每次都要写的那堆固定起手代码。

- 生活中的例子

写信开头的“你好，见信如晤”那套。

- 程序员真正想表达什么

重复出现的样板起始代码。

- AI 会翻译成什么开发需求

用现成 boilerplate 省去重复起步。

✓ 什么时候需要

新项目起步、想少写重复时。

✗ 什么时候不需要

已有现成结构时。

- 没有它会怎样

每次都从零敲一堆重复代码。

小白应该怎么告诉 AI

帮我用一套 boilerplate 快速起步。

👉 起手那套，别每次重写。👈

Scaffold

Scaffold



一句话理解

一键生成项目骨架，把架子先搭好。

- **生活中的例子**

盖房子先把脚手架和框架搭起来。

- **程序员真正想表达什么**

自动生成项目基础结构。

- **AI 会翻译成什么开发需求**

用脚手架命令生成项目骨架。

✓ **什么时候需要**

新项目要快速搭架子时。

✗ **什么时候不需要**

已有项目结构时。

- **没有它会怎样**

从零建目录结构又慢又易漏。

小白应该怎么告诉 AI

帮我 scaffold 出一个项目骨架。

﹂ 架子先搭好，再填内容。 ﹂

Bug Report

Bug Report



一句话理解

报告一个哪里坏了的问题单。

- 生活中的例子

报修单：哪坏了、怎么复现。

- 程序员真正想表达什么

描述缺陷及复现步骤的报告。

- AI 会翻译成什么开发需求

按模板提交 bug report。

✓ 什么时候需要

发现问题要反馈时。

✗ 什么时候不需要

没有问题时。

- 没有它会怎样

问题说不清，修的人抓瞎。

小白应该怎么告诉 AI

帮我写一份 bug report，说明__怎么出错的。

“说清怎么坏，才好修。”

Feature Request

Feature Request



一句话理解

提一个“希望能加这个功能”的请求。

- 生活中的例子

意见箱里写“希望能加个夜间模式”。

- 程序员真正想表达什么

请求新增功能的提案。

- AI 会翻译成什么开发需求

按模板提交 feature request。

✓ 什么时候需要

想建议新功能时。

✗ 什么时候不需要

无新需求时。

- 没有它会怎样

好点子没处提，白白流失。

小白应该怎么告诉 AI

帮我写一个 feature request，建议加__。

“想要啥功能，提出来。”

Push

Push



一句话理解

把本地的改动传到云端仓库。

- **生活中的例子**

把家里写好的稿子上传到共享网盘。

- **程序员真正想表达什么**

将本地提交上传到远程仓库。

- **AI 会翻译成什么开发需求**

push 把本地提交同步到远程。

✓ **什么时候需要**

改完要同步到云端时。

✗ **什么时候不需要**

只在本地改时。

- **没有它会怎样**

改动只在本地，别人看不到。

小白应该怎么告诉 AI

帮我把改动 push 到远程仓库。

ゝ 写完往上一推，云端就有了。ゝ

Pull

Pull



一句话理解

把云端最新的改动拉到本地。

- **生活中的例子**

把网盘里同事更新的稿子下载下来。

- **程序员真正想表达什么**

从远程拉取并合并最新改动。

- **AI 会翻译成什么开发需求**

pull 获取远程最新代码。

✓ **什么时候需要**

要同步别人最新改动时。

✗ **什么时候不需要**

独自离线改时。

- **没有它会怎样**

本地是旧的，容易冲突。

小白应该怎么告诉 AI

帮我 pull 一下远程最新代码。

“先拉最新，再动手。”

Remote

Remote



一句话理解

云端那个远程仓库地址。

- **生活中的例子**

共享网盘的那个总地址。

- **程序员真正想表达什么**

本地关联的远程仓库。

- **AI 会翻译成什么开发需求**

配置 remote 指向云端仓库。

✓ **什么时候需要**

要和云端同步时。

✗ **什么时候不需要**

纯本地时。

- **没有它会怎样**

本地不知道该往哪同步。

小白应该怎么告诉 AI

帮我把 remote 指向这个云端仓库。

“remote，是云端那头。”

Origin

Origin



一句话理解

你默认的那个远程仓库的名字。

- 生活中的例子

网盘存的“主地址”，默认就用它。

- 程序员真正想表达什么

默认远程仓库的别名。

- AI 会翻译成什么开发需求

把 origin 作为默认远程推拉。

- ✓ 什么时候需要

日常推拉代码时。

- ✗ 什么时候不需要

无远程时。

- 没有它会怎样

不知道默认往哪推拉。

小白应该怎么告诉 AI

帮我把代码 push 到 origin。

“origin，就是默认那头。”

术语索引

按中文（拼音 / 笔画大致排序）

Agent	Agent	019	Push	Push	081
Anthropic API	Anthropic API	008	RAG	Retrieval-Augmented Generation	023
Apache	Apache License	067	README	README	064
Boilerplate	Boilerplate	077	Reasoning	Reasoning	018
Branch	Branch	046	Rebase	Rebase	062
Bug Report	Bug Report	079	Release	Release	050
ChatGPT	ChatGPT	001	Remote	Remote	083
Cherry-pick	Cherry-pick	061	Repository	Repository	042
Claude Code	Claude Code	003	Roadmap	Roadmap	073
Clone	Clone	044	Scaffold	Scaffold	078
Codex	Codex	002	Semantic Version	Semantic Versioning	072
Commit	Commit	045	Skill	Skill	021
Conflict	Conflict	060	Star	Star	052
Contributor	Contributor	054	System Prompt	System Prompt	012
Cursor	Cursor	004	Tag	Tag	051
Diff	Diff	058	Template	Template	076
Discussion	Discussion	055	Token	Token	026
Embedding	Embedding	024	Tool	Tool	016
Feature Request	Feature Request	080	User Prompt	User Prompt	013
Fork	Fork	043	VS Code	VS Code	039
Function Calling	Function Calling	017	Watch	Watch	053
Gemini	Gemini	005	Wiki	Wiki	056
GitHub	GitHub	041	Windsurf	Windsurf	006
GitHub Actions	GitHub Actions	057	Workflow	Workflow	020
IDE	IDE	038	上下文	Context	014
Issue	Issue	049	代码审查	Code Review	032
Label	Label	075	代码生成	Code Generation	031
License	License	065	单元测试	Unit Test	035
Lint	Lint	037	向量数据库	Vector Database	025
MCP	Model Context Protocol	022	图像理解	Image Understanding	030
MIT	MIT License	066	多模态	Multimodal	027
Main	Main / Master	063	大模型	Large Language Model	010
Markdown	Markdown	040	开源	Open Source	068
Memory	Memory	015	模型	Model	009
Merge	Merge	047	版本号	Version Number	071
Milestone	Milestone	074	语音识别	Speech Recognition	029
OCR	OCR	028			

索引 · INDEX (续)

按英文 (A-Z)

Agent Agent	019	Milestone Milestone	074
Anthropic API Anthropic API	008	MIT License MIT	066
Apache License Apache	067	Model 模型	009
Boilerplate Boilerplate	077	Model Context Protocol MCP	022
Branch Branch	046	Multimodal 多模态	027
Bug Report Bug Report	079	OCR OCR	028
ChatGPT ChatGPT	001	Open Source 开源	068
Cherry-pick Cherry-pick	061	OpenAI API OpenAI API	007
Claude Code Claude Code	003	Origin Origin	084
Clone Clone	044	Patch Patch	059
Closed Source 闭源	069	Prompt Prompt	011
Code Generation 代码生成	031	Pull Pull	082
Code Review 代码审查	032	Pull Request Pull Request	048
Codex Codex	002	Push Push	081
Commit Commit	045	README README	064
Conflict Conflict	060	Reasoning Reasoning	018
Context 上下文	014	Rebase Rebase	062
Contributing Guide 贡献指南	070	Refactor 重构	034
Contributor Contributor	054	Release Release	050
Cursor Cursor	004	Remote Remote	083
Debug 调试	033	Repository Repository	042
Diff Diff	058	Retrieval-Augmented Generation RAG	023
Discussion Discussion	055	Roadmap Roadmap	073
Embedding Embedding	024	Scaffold Scaffold	078
Feature Request Feature Request	080	Semantic Versioning Semantic Version	072
Fork Fork	043	Skill Skill	021
Function Calling Function Calling	017	Speech Recognition 语音识别	029
Gemini Gemini	005	Star Star	052
GitHub GitHub	041	System Prompt System Prompt	012
GitHub Actions GitHub Actions	057	Tag Tag	051
IDE IDE	038	Template Template	076
Image Understanding 图像理解	030	Token Token	026
Integration Test 集成测试	036	Tool Tool	016
Issue Issue	049	Unit Test 单元测试	035
Label Label	075	User Prompt User Prompt	013
Large Language Model 大模型	010	Vector Database 向量数据库	025
License License	065	Version Number 版本号	071
Lint Lint	037	VS Code VS Code	039
Main / Master Main	063	Watch Watch	053
Markdown Markdown	040	Wiki Wiki	056
Memory Memory	015	Windsurf Windsurf	006

写在最后

黑话不是为了拒人于门外，它只是行业里的省事说法。当你能把它翻译成人话，你就已经能和 AI、和程序员平等对话了。

这本手册的野心很小：让你在听到一个词时，不再点头假装懂，而是能说出“哦，原来它就是那个意思”。它的野心也很大：让不懂技术的人，也能用 AI 把想法做成真正的产品。

如果某个词的解释你觉得还能更清楚，欢迎在 GitHub 上提出。这本书会一直更新。

本册是《AI 开发黑话翻译手册》三册中的第三册。三册合起来，覆盖从产品想法、到开发部署、再到 AI 编程与 GitHub 的完整旅程。

翻译逻辑始终不变：**程序员语言 → 普通人语言 → AI 能执行的开发需求语言**。